Programming with Artificial Intelligence

Hajdú Patrik Zsolt

Table of Contents

1. The Paradigm Shift in Software Engineering Information Retrieval 2
   1.1 Static Document Analysis (Ctrl + F) vs. Semantic Context Interpretation 2
   1.2 Community Knowledge Bases vs. Generative Synthesis 2
2. Personal Experiences with Prompting: "Vibe Coding" in Practice 3
   2.1. Zero-shot prompting: Immediate answers, but limited coding 3
   2.2. Few-shot prompting: The effective, but context-window demanding golden mean 3
   2.3. Chain of Thought (CoT): Collaborative thinking and engineering deduction 4
3. Leveraging the Latest Technologies: Enterprise Security, Agents, and Fine-tuning 5
   3.1. RAG: Enterprise data security and the evolution of "Ctrl + F" 5
   3.2. Beyond the chat window: MCP Servers and AI Agents 5
   3.3. LoRA and Fine-tuning: When general knowledge runs out 6
4. Concluding Thoughts: AI is not the enemy, but the next infrastructure 7
5. References and Tools Used 8
   5.1. AI Models and Tools Used 8
   5.2. Literature, Articles, and Documentation 8

1. The Paradigm Shift in Software Engineering Information Retrieval

In the public consciousness, software development is equated with continuous coding, but in reality, a significant portion of our working hours is dedicated to something entirely different: debugging, browsing documentation, and retrieving appropriate information. However, the emergence of Artificial Intelligence (AI), and particularly Large Language Models (LLMs), has fundamentally changed the rules of the game. The first chapter of this case study explores how online and locally hosted chat-based AI assistants are transforming our daily programming routines.

1.1 Static Document Analysis (Ctrl + F) vs. Semantic Context Interpretation

Traditional search, known to all of us through the "Ctrl + F" keyboard shortcut, is essentially simple pattern matching. Although from an algorithmic standpoint (as an O(N) complexity operation) it reliably finds the typed word, it completely fails to understand the context. If we are looking for a bug in a multi-hundred-page API documentation, but the description refers to a "client" while we search for a "user", Ctrl + F will fail us.

In contrast, if we provide the exact same documentation to an AI model (e.g., Gemini or a local Llama 3), the machine interprets the words at a semantic level, based on their meaning. The practical advantage is massive: if we drop in an error message, the AI not only locates it in the documentation but also "understands" the causal relationships and immediately proposes a solution. This saves an enormous amount of time otherwise spent on investigative work.

1.2 Community Knowledge Bases vs. Generative Synthesis

For years, Stack Overflow, GitHub, or GeeksforGeeks were a programmer's best friends when they got stuck. However, the biggest issue with these forums is that the found solution must always be adapted to our own project. Even if we find a brilliant code snippet, we have to manually rewrite the variable names, handle version discrepancies, and adjust the business logic. This is an exhausting process fraught with potential errors.

Chat-based AI assistants save us this manual intermediate step. An LLM not only "retrieves" the answer but, when used correctly, is capable of immediately generating the response customized to our unique environment, utilizing our specific variable names and frameworks.

2. Personal Experiences with Prompting: "Vibe Coding" in Practice

The first chapter of the case study explored how online and locally hosted chat-based AI assistants are transforming our daily programming routines and why they are more efficient than traditional search. Now, let us move to more practical waters: how should we actually talk to these models if we want to write functional code?

Prompting in practice is not black magic, but a highly conscious form of communication. Based on my personal experiences–and the accounts of many of my fellow Computer Engineering students at the Budapest University of Technology and Economics – the success of collaborating with a chat-based AI depends on knowing which strategy to deploy and when. Let's examine the three most important approaches!

2.1. Zero-shot prompting: Immediate answers, but limited coding

When we simply drop a question without any prior context or examples, it is called zero-shot prompting. As an IT professional, I have found this method brilliant when I want to understand a new "buzzword" or seek an answer to a general theoretical concept where the AI does not need to know the environment of my project (for example: "What is the difference between TCP and UDP protocols?").

However, for coding tasks, this is usually a terrible choice. Why? Because without deeper context, the AI loses the thread after the first one or two prompts. It does not know exactly what it should do within our specific system, so it starts to "hallucinate" – confidently generating faulty, non-existent functions or entirely irrelevant logic.

2.2. Few-shot prompting: The effective, but context-window demanding golden mean

Generally, this is the best choice during daily development. Here, we do not just ask; we provide a precise description of the task and support the prompt with 1-5 concrete examples. We supply the environment of the current query, characteristic information, and the validation rules the output must meet–we may even paste direct code snippets as templates.

This method is highly effective during coding and testing because the AI recognizes the pattern. But it has a massive drawback: with this detailed method, we can fill up the model's memory, the Context Window, very quickly. As the injected codes and rules accumulate, the AI loses its "memory" of previous instructions. It begins to guess, hallucinates again, and we are often forced to re-analyze the past conversation with it, or start completely from scratch with a clean slate in a new chat window.

2.3. Chain of Thought (CoT): Collaborative thinking and engineering deduction

Personally, this is the path I prefer to take. The essence of the Chain of Thought method is that we do not immediately demand the final, complete code from the AI. Instead, we build the answer slowly, based on curiosity. We engage in a dialogue: we ask, "what if we used this approach?", or "why do we do it this way, and why not that way?", until we arrive at a stable answer.

The greatest advantage of this is that it does not merely provide a drop-in solution; it also shows the path of how we got there. Although this approach is undeniably more time-consuming, it is far more informative and educational. It aids in mastering the engineering mindset, as the model justifies its steps and logically deduces their correctness. This thoughtfulness not only reduces the number of errors but also deepens our own understanding, which incredibly accelerates future collaborative work.

3. Leveraging the Latest Technologies: Enterprise Security, Agents, and Fine-tuning

As we dig deeper into AI-assisted software development, we soon realize that simple browser-based chat windows have their limits. In a real enterprise project, we cannot paste the company's most closely guarded source code into a global model, nor can we expect a complex refactoring from a single prompt. The latest industry trends provide answers to exactly these engineering challenges.

3.1. RAG: Enterprise data security and the evolution of "Ctrl + F"

The biggest hurdle to enterprise AI adoption is data security. At most companies, it is strictly forbidden to upload internal documents, API keys, or source code to public AI models (like ChatGPT), lest the model learns them into its own global weights and later leaks them to others.

This is where RAG (Retrieval-Augmented Generation) comes into play, serving as a brilliant abstraction layer. The essence of RAG is that it separates the AI's logical capabilities from its memory. When we upload company documentation to our own closed system, RAG does not train the model on it. Instead, it vectorizes the texts (turns them into numbers) and stores them in a local database. When we ask something, the system first retrieves the most relevant paragraphs (this is the modern, semantic version of the classic Ctrl + F), and then temporarily places this small text snippet into the model's context window.

The engineering elegance lies in the fact that the AI can provide a logical, coherent answer based on our most secret company data without actually learning from it. As soon as the answer is generated, the data disappears from the model's "short-term memory," guaranteeing maximum protection.

3.2. Beyond the chat window: MCP Servers and AI Agents

If RAG solves reading, MCP (Model Context Protocol) and AI Agents solve taking action.

What is MCP? The Model Context Protocol is a brand-new, open standard (introduced by Anthropic) that builds a bridge between the AI and our local development environment. It is essentially a collection of servers that allow the AI (under strict permission management) to read into our local files, query a database, or even run a bash script.

AI Agents in practice: While a traditional AI Chat passively waits for our question, an AI Agent is proactive. If we issue a complex task (e.g., "Create a new endpoint in the backend, and write the tests for it"), the agent breaks this large problem down into small, logical subtasks. Then–using MCP servers and the appropriate tools–it begins iterative cycles: it writes the code, runs the test, sees the error message, rethinks the problem, and rewrites the code.

As a programmer, this is currently the highest quality code generation option available. In modern development environments (like Cursor or the appropriate VS Code extensions), we can watch live as the agent "thinks," navigates through our folders, and before saving anything, shows us exactly which lines it wants to modify in our files via a clean Git Diff view.

3.3. LoRA and Fine-tuning: When general knowledge runs out

There are cases when, despite providing perfect prompts and using RAG, the model is simply not specific enough. Suppose our project aims to create software that distinguishes rare snake species based on a unique dataset we collected (or perhaps we need to maintain code in a very rare, obsolete programming language). The general AI models on the market are not prepared for this.

This is when Fine-tuning is required, specifically its most cost-effective engineering solution, LoRA (Low-Rank Adaptation). Here, we are no longer just "whispering" to the model through the context window; we are actually modifying its internal parameters (weights). We purposefully train a strong but general model (e.g., an open-source Llama) on our own specific dataset. The result will be a specialized AI that might lose its ability to write us a poem about spring, but in return, it delivers pinpoint, uncompromising performance in the single task our project requires.

4. Concluding Thoughts: AI is not the enemy, but the next infrastructure

Throughout writing this case study, as we reviewed the limitations of traditional information retrieval, the engineering pitfalls of prompting, and modern architectural solutions like RAG or MCP servers, one thing became crystal clear: artificial intelligence is currently the most important catalyst in software development.

We often hear fear within the profession–especially in junior or university circles–that AI will take away programmers' jobs. However, looking at it purely from a technological perspective, under the current circumstances, we should not oppose AI; we must integrate it. AI is an innovation milestone exactly like the dawn of the internet or the proliferation of Cloud Computing a decade ago.

When companies started migrating from their own server rooms (on-premise) to AWS or Azure, system administrators did not become unemployed. Their roles transformed: instead of untangling cables, they began designing cloud architectures (DevOps). The exact same paradigm shift is happening in programming right now. AI is the new abstraction layer.

As IT professionals, we must find the channels where we can deploy AI as a tool:

1. **Expanding Knowledge Bases:** Use Chain of Thought prompting to help ourselves understand a complex network architecture or an unfamiliar framework.
2. **Maximizing Productivity:** Offload monotonous, repetitive tasks (writing boilerplate code, generating unit tests, format conversions) to AI Agents, so we only need to focus on high-level business logic and system design.
3. **Shipping Features:** Do not just use AI; integrate it into your own projects! A custom-developed software with an integrated RAG module or intelligent data processing provides an immediate market advantage.

The essence of engineering work has never been about memorizing syntax, but about problem-solving. The IT professional who learns to "orchestrate" AI models and integrate them into their workflows will not lose their job; rather, they will become irreplaceable in the future software industry.

5. References and Tools Used

To prepare this case study, understand the technological concepts (Prompting, RAG, MCP, LoRA), and structure the text, I utilized the following resources and generative AI tools.

5.1. AI Models and Tools Used

Google Gemini 3.1 Pro: Used during the research phase for iterative discussion of concepts and the engineering deduction of outline points (RAG and MCP operation).

5.2. Literature, Articles, and Documentation

Prompt Engineering and LLM interactions: DAIR.AI (2024). *Prompt Engineering Guide*. Detailed description of the engineering background behind Zero-shot, Few-shot, and Chain of Thought strategies. Available at: promptingguide.ai

RAG (Retrieval-Augmented Generation): Pinecone (2024). *What is Retrieval-Augmented Generation (RAG)?* Practical guide to integrating vector databases and the RAG pipeline. Available at: pinecone.io/learn/retrieval-augmented-generation/

Model Context Protocol (MCP) and Agents: Anthropic (2024). *Model Context Protocol Specification*. The official technical documentation and implementation guide for the open standard. Available at: modelcontextprotocol.io

LoRA (Low-Rank Adaptation): IBM (2024). *What is LoRA?* IBM Think Topics. A comprehensive industry and enterprise summary of the technology's practical application, perfectly complementing the mathematical foundations. Available at: ibm.com/think/topics/lora